

Abstract Phase III

Projekt Cloud Programming

Jonas Habel
Matrikelnummer: 9227109

17. Februar 2026

Dieses Abstract fasst die Umsetzung und Reflexion des Projekts „Cloud Programming“ in Phase III zusammen. Ziel war die Bereitstellung einer Unternehmenswebsite, die hochverfügbar, global erreichbar und automatisch skalierbar ist. Die technische Umsetzung erfolgte in Azure mit Terraform als Infrastructure-as-Code-Ansatz.

Making-of

Von der Zielsetzung zur Methodik

Im Mittelpunkt stand nicht nur ein funktionierendes Deployment, sondern ein reproduzierbarer Betriebsprozess. Dafür wurden die Anforderungen in technische Teilziele übersetzt und anschließend als deklarative Infrastruktur modelliert. Die Methodik war iterativ:

1. Definition der Qualitätsziele Verfügbarkeit, Latenz und Skalierbarkeit.
2. Auswahl der Azure-Dienste und Modellierung in Terraform.
3. Aufbau eines automatisierten Build- und Deploymentablaufs.
4. Verifikation über lokale Läufe und Pipeline-Ausführungen.

Die Anwendung selbst wurde bewusst simpel gehalten (statische HTML in einem NGINX-Container), um den Fokus auf Cloud-Architektur, Betrieb und Automatisierung zu legen.

Quellen/Ressourcen/Software

Folgende Ressourcen wurden für die Umsetzung genutzt:

- GitHub (Codeverwaltung) und GitHub Actions (Build/Push-Workflow).
- Docker mit `nginx:1.27-alpine` als Container-Basis.
- Terraform (`azurerm`, `random`) mit Remote State in Azure Storage.
- Azure Container Registry als Artefakt-Repository.
- Azure Container Apps in West Europe und North Europe.
- Azure Front Door als globaler Entry Point mit Routing und Failover.
- Azure Key Vault, Service Principal und Managed Identity für Zugriff und Secrets.

- Microsoft Learn: Azure Front Door und Microsoft Learn: Azure Container Apps.
- Terraform AzureRM Provider Dokumentation und Azure Login Action.

Breakdown der Umsetzung

Die Umsetzung wurde in fünf Arbeitspakete aufgeteilt:

1. **Infrastrukturmodell:** Resource Group, Container Registry, zwei Container-App-Environments und Log-Analytics-Workspaces wurden als Terraform-Ressourcen angelegt.
2. **Anwendungslayer:** Zwei Container Apps (Primary/Secondary) wurden mit identischem Image, externer Ingress-Konfiguration und Autoscaling (1–10 Replikas, HTTP-Regel) bereitgestellt.
3. **Globales Routing:** Azure Front Door verbindet beide Regionen; Prioritäten und Health Probes ermöglichen Failover.
4. **Sicherheit und Zugriff:** Managed Identity mit `AcrPull` ersetzt statische Registry-Credentials; Secrets werden über Key Vault bezogen.
5. **Deploymentprozess:** Das Skript `terraform-local.cmd` automatisiert Key-Vault-Read, Azure-Login, Image-Build/Push und Terraform-Apply; die Pipeline automatisiert den Build/Push-Teil.

Diese Arbeitsteilung war wichtig, weil Architektur, Security und Betrieb in Cloud-Projekten eng zusammenhängen. Insbesondere die Trennung zwischen Artefaktfluss (Image-Build/Push) und Infrastrukturfluss (Terraform-State und Apply) hat die Umsetzung transparenter gemacht und Fehleranalyse vereinfacht. Bei Änderungen konnte dadurch gezielt entschieden werden, ob ein reiner Image-Rollout ausreicht oder eine Infrastrukturänderung notwendig ist.

Reflexion

Das erreichte Ergebnis ist funktional und technisch konsistent: Die Website wird containerisiert ausgeliefert, über einen globalen Endpunkt erreichbar gemacht und in zwei Regionen betrieben. Die Infrastruktur ist vollständig im Code beschrieben und daher reproduzierbar.

Damit entspricht das Ergebnis dem Ziel der Arbeit weitgehend. Nicht vollständig erfüllt ist jedoch der Nachweisgrad:

- Die Anwendung ist ein Demonstrator und nicht produktionsnah in der Fachlogik.
- Skalierungs- und Failoververhalten sind konfiguriert, aber noch nicht durch automatisierte Lasttests quantitativ belegt.
- Der CI-Workflow automatisiert aktuell den Artefaktfluss, nicht den vollständigen End-to-End-Infrastruktur-Deploy.

Aus dem Bearbeitungsprozess lassen sich folgende Schlüsse ziehen:

1. **IaC verbessert Nachvollziehbarkeit und Stabilität.** Änderungen sind versionierbar und reproduzierbar, manuelle Drift wird reduziert.
2. **Security-by-Design lohnt sich früh.** Key Vault, Rollenmodell und Managed Identity vereinfachen den Betrieb deutlich.
3. **Automatisierung endet nicht beim Build.** Für höhere Reife sind automatisierte Deploy-, Last- und Resilienztests der nächste zwingende Schritt.

Ein zusätzlicher Lernpunkt war die praktische Bedeutung sauberer Variablen- und Namenskonzepte in Terraform. Die Parametrisierung mit Regionen, Replikagrenzen und optional bestehender Registry hat gezeigt, wie stark Wartbarkeit von konsistentem Infrastrukturdesign abhängt. Für kommende Arbeiten nehme ich mit, technische Anforderungen früher in messbare Akzeptanzkriterien zu übersetzen, damit die Zielerreichung nicht nur qualitativ, sondern auch quantitativ abgesichert ist.

Fazit

Das Projekt zeigt, dass sich die geforderten Cloud-Eigenschaften mit überschaubarem Umfang strukturiert umsetzen lassen: mehrregionale Bereitstellung, globales Routing, deklarative Infrastruktur und automatisierter Build-Prozess. Die Zielsetzung wurde damit inhaltlich erreicht. Als nächster Reifegrad sind End-to-End-Automatisierung und belastbare Betriebsnachweise (Load und Failover) sinnvoll, um die Lösung produktionsnäher abzusichern.